I. Week 1 FUNDAMENTALS OF INFRASTRUCTURE

Date:02/11/2026
Prepared by: Rustamzada Ruslan

Summary of Key Activities

- **This report summarises the infrastructure components implemented during Week 1, the challenges encountered, and the solutions applied.**
- **Key milestones of the project phases have been completed.**

In this report pages each part has been specified and divided into “mini projects”

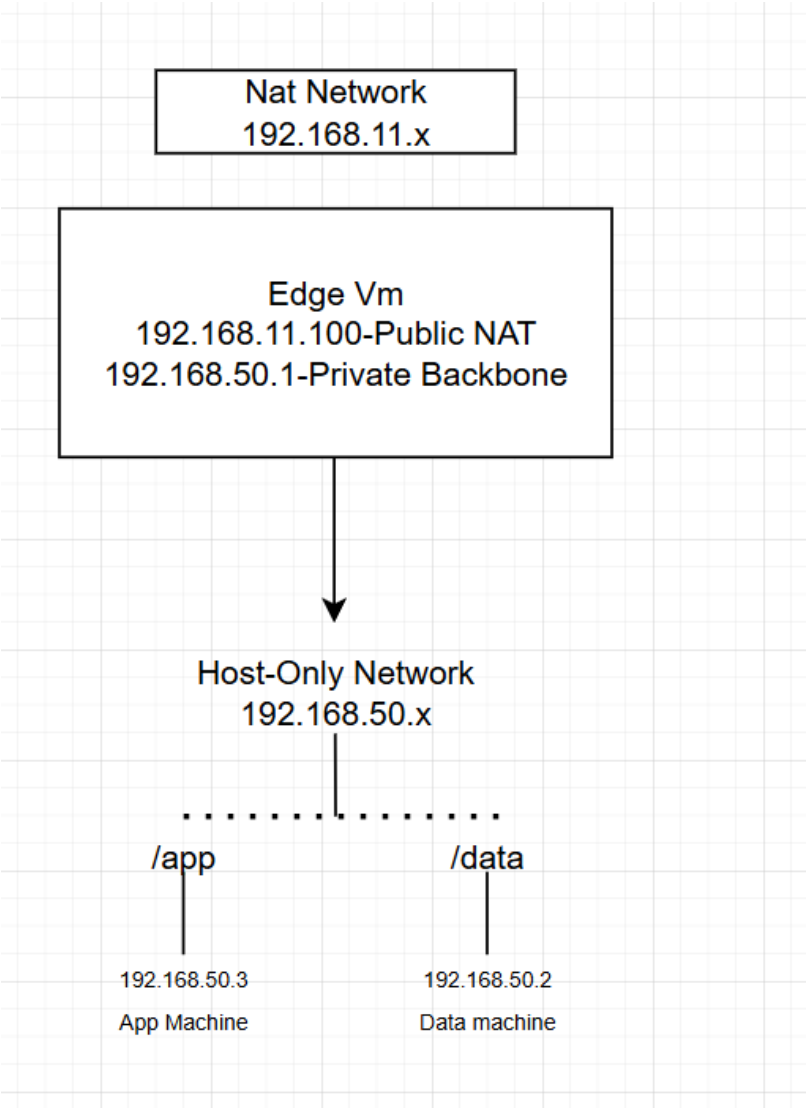
Project Progress and Milestones

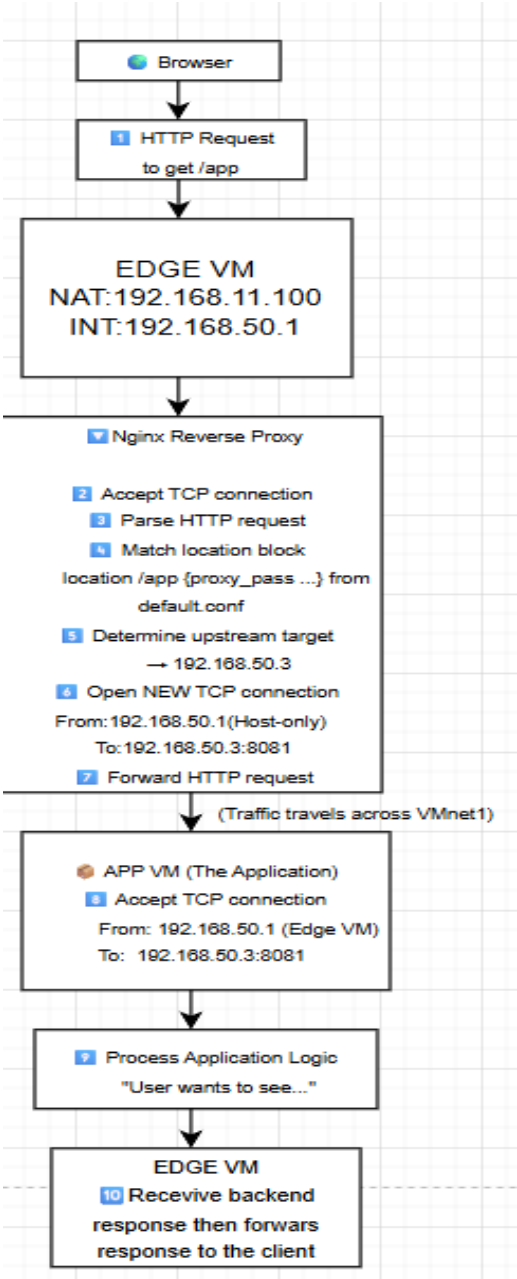
- **Internal Network (Virtual LAN): [Completed]**
- **Reverse Proxy and Virtual IP: [Completed]**
- **Internal Web Servers: [Completed]**
- **Live Monitoring Dashboard: [Completed]**
- **Active access and Error Logging: [Completed]**

Accomplishment

During this week, 4 virtual machines were created and built an internal network architecture consisting of:

I. **The EDGE VM is the only machine with external network access. All other machines are isolated from the internet but can communicate internally with each other.**
II. **Using the EDGE machine, I deployed NGINX on the internal servers**
III. **Created Reverse Proxy and Host-Only Network. With the help of these 2 tools, the machines “knew” each other and could talk to each other can “curl” each other's web servers. Also, with the reverse proxy only writing EDGE’s IP and the machine’s web server’s name (which can’t reach the outside) can easily be seen through the browser**


Nat Network
192.168.11.x
Edge Vm
192.168.11.100-Public NAT
192.168.50.1-Private Backbone
Host-Only Network
192.168.50.x
/app
/data
192.168.50.3
App Machine
192.168.50.2
Data machine

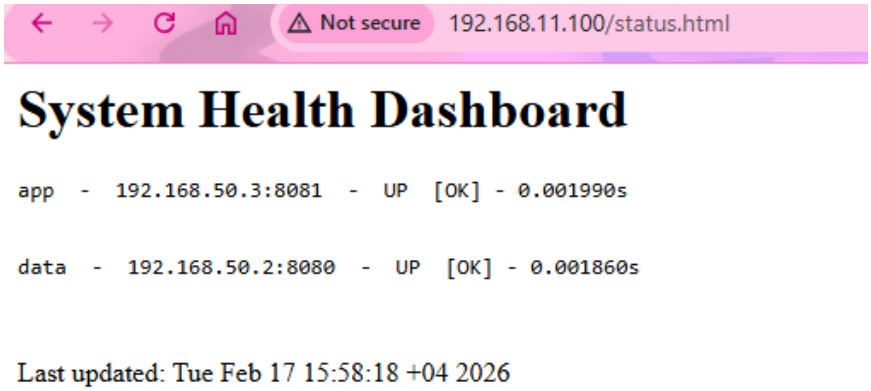


Also, I edited the default.conf file of the nginx container that helps to work properly of my reverse-proxy “engine”, and here is the edited version of the default.conf file:

```
 log_format monitor '$remote_addr - $request_uri - $status - $request_time';
server {
    listen       80;
    listen  [::]:80;
    server_name  localhost;
    access_log  /var/log/nginx/access.log;
    access_log /var/log/nginx/monitor-alert.log monitor;
    error_log /var/log/nginx/error.log warn;
    location / {
        root   /usr/share/nginx/html;
        index  index.html index.htm;
    }
    }
    location /data {
        proxy_pass http://192.168.50.2:8080/;
    }

    location /app {
        proxy_pass http://192.168.50.3:8081/;
    }

    #error_page  404              /404.html;

    # redirect server error pages to the static page /50x.html
    #
    error_page   500 502 503 504  /50x.html;
    location = /50x.html {
        root   /usr/share/nginx/html;
    }

    # proxy the PHP scripts to Apache listening on 127.0.0.1:80
    #
    #location ~ \.php$ {
    #    proxy_pass   http://127.0.0.1;
    #}

    # pass the PHP scripts to FastCGI server listening on 127.0.0.1:9000
    #
    #location ~ \.php$ {
    #    root           html;
    #    fastcgi_pass   127.0.0.1:9000;
    #    fastcgi_index  index.php;
    #    fastcgi_param  SCRIPT_FILENAME  /scripts$fastcgi_script_name;
    #    include        fastcgi_params;
    #}

    # deny access to .htaccess files, if Apache's document root
    # concurs with nginx's one
    #
    #location ~ /\.ht {
    #    deny  all;
    #}
}
```

This part explains that phase 4(matching the block)

IV. Active access and error logging/Live monitoring dashboard:



For understanding how this dashboard work properply, we need to dive into these 2 parts: Scripts and the Dockerfile

1) This script is the core component of the live dashboard. It runs in the background and updates the dashboard every 5 seconds.

```bash
#!/bin/bash
# Start a background loop to update the status page every 5 seconds
(
  while true; do
    /usr/local/bin/check_status.sh
    sleep 5
  done
) &
# Start Nginx in the foreground (this keeps the container alive)
nginx -g "daemon off;"
```

2) This script is the brain of the dashboard. Basically, every 5 seconds it ping the all services and check theirs' health and automatically create html file(if it has not been created) and updates the file.

```bash
#!/bin/bash

# Define where to save the HTML report
HTML_FILE="/usr/share/nginx/html/status.html"

# Start the HTML file
echo "<html><head><title>System Status</title><meta http-equiv='refresh' content='5'></head><body>" > $HTML_FILE
echo "<h1>System Health Dashboard</h1><pre>" >> $HTML_FILE

# Print Header to Terminal
printf "%-10s %-20s %-10s\n" "SERVICE" "ADDRESS" "STATUS"
printf "%-10s %-20s %-10s\n" "-------" "-------" "------"

# Define your backend targets
# Format: Name:IP:Port
targets=("app:192.168.50.3:8081" "data:192.168.50.2:8080")

for target in "${targets[@]}"; do

    # Split target definition into name, IP, and port
    # Example input: "app:192.168.50.3:8081"
    IFS=':' read -r name ip port <<< "$target"

    # Perform HTTP health check
    # -s  → silent
    # -o /dev/null → discard body
    # -w → print HTTP code and total response time
    # --max-time 2 → timeout protection (2 seconds)
    result=$(curl -s -o /dev/null -w "%{http_code} %{time_total}" --max-time 2 "http://$ip:$port/index.html")

    # Extract HTTP code and response time
    read -r code time <<< "$result"
    # Define status based on the code
    case "$code" in
        200)
                status="200 OK - ${time}s";;
        404)
                status="404 Not Found";;
        502)
                status="502 Bad Gateway";;
        503)
                status="503 Service Unavailable";;
        000)
                status="000 Connection Refused";;
        *)
                status="$code Unknown Error";;
    esac

    # Determine service state
    if [ "$code" -eq 200 ]; then

        # Performance classification
        if (( $(echo "$time < 0.5" | bc -l) )); then
            status="UP  [OK] - ${time}s"
        else
            status="UP  [SLOW] - ${time}s"
        fi

    elif [ "$code" -ne 200 ]; then
        # Log alert event
        echo "[$(date '+%Y-%m-%d %H:%M:%S')] ALERT: $name returned $status">>/var/log/nginx/monitor-alert.log
        status="Down [$code]"
    fi

    # Show in Terminal
    printf "%-10s %-20s %-10s\n" "$name" "$ip:$port" "$status"

    # Write to Web Page
    echo "$name  -  $ip:$port  -  $status" >> $HTML_FILE
    echo "<br>" >> $HTML_FILE
done

echo "</pre><p>Last updated: $(date)</p></body></html>" >> $HTML_FILE
```

3) In this image, as you see, it is obvious. Before deploying the nginx server, which is built to serve as our reverse proxy, also copy and run the scripts that we can use for the live dashboard. With the help of this image, we automatically build these:

- Reverse-proxy

- Live dashboard
- Log file (even though the log files were not created with the help of this section of the default.conf it created automatically while building this image)

```
 log_format monitor '$remote_addr - $request_uri - $status - $request_time';
server {
    listen       80;
    listen  [::]:80;
    server_name  localhost;
    access_log  /var/log/nginx/access.log;
    access_log /var/log/nginx/monitor-alert.log monitor;
    error_log /var/log/nginx/error.log warn;
```

```
FROM nginx:latest
#1.Copy Configs
COPY default.conf /etc/nginx/conf.d/default.conf
COPY html/ /usr/share/nginx/html/
#2. Copy the scripts
COPY check_status.sh /usr/local/bin/check_status.sh
COPY boot.sh /usr/local/bin/boot.sh
#3. Make the scripts executable
RUN chmod +x /usr/local/bin/check_status.sh /usr/local/bin/boot.sh
#4. Set the Timezone[Asia/Baku GMT +04]
ENV TZ=Asia/Baku
RUN ln -snf /usr/share/zoneinfo/$TZ /etc/localtime && echo $TZ > /etc/timezone
#5. Fix the logs
RUN rm -f /var/log/nginx/access.log /var/log/nginx/error.log
#6. Use the script at startup
CMD ["/usr/local/bin/boot.sh"]
```

Challenges and Issues

- **Private Backbone issues and their solution**
  - After creating a new subnet for internal IP, it needs to add another new local network for EDGE(think like a bicycle for the movement, it needs 2 wheels), one of VMnet8(External/NAT) other is VMnet1(Host-only/Internal network). In this way, EDGE machines can speak to each other through the new subnet(+security), and other machines can curl each other's web servers, even the EDGE machines' too.
  - Also add firewall-cmd --zone=trusted --add-source=IP_WITH_NEWSUBNET/24 also firewall-cmd --add-port=specified range/tcp --permanent then reload the firewall-cmd
  - These firewall rules allowed the new subnet to be trusted and enabled the required TCP ports, ensuring proper communication between the machines.
- **Fix the log files**
  - After editing the default conf for automatically created log files, it just creates the symlink, which cannot be watched live through the "tail -f" command. The problem I have is the location confusion. In docker image, I wrote "COPY default.conf /etc/nginx/conf.d/default.conf", and that causes the location conflict. When I changed the location to the

"/root/edge/image/default.conf", it worked perfectly because the location is true.

- **Making the containers alive after rebooting with systemd**
  - After creating nginx services for all machines, another problem appears. The life span of the containers after rebooting.
  - The solution is so easy. In the /etc/systemd/system directory, I wrote this command "podman generate systemd --new --name container name --files". So it creates the container service that can easily start and enable, or if some updates are needed, just stop the service, then start the service or reload the service for sourcing/updating the new rules/commands.
- **Podman stop/start problem**
  - After creating the container services, it causes another problem. When I wrote "podman stop container then podman start container", it gets error, because I wrote before "--new" in the systemd part. Think of it like this:
    - Podman (runtime-level control)
      - Manages containers directly using "podman start/stop/rm
    - Systemd (service-level control)
      - Manage services declaratively
      - Uses unit files
      - Ensures the desired state with "systemctl start/stop/restart"

!!!ONLY ONE CONTROLLER SHOULD OWN THE CONTAINER LIFECYCLE!!!

- The "--new" flag means:
  - The container is not treated as a permanently existing object.
  - The systemd unit file contains instructions to:
    - Create the container when starting the service
    - Remove the container when stopping the service
- In this model:

```
systemctl start container.service
→ podman run ...

systemctl stop container.service
→ podman stop
→ podman rm
```

This container is ephemeral.

- **WHY MIXING COMMANDS CAUSES PROBLEMS**

- podman stop container. I am bypassing the systemd controller.
- Systemd still assumes:
  - It owns the service
  - It defines how the container should exist
  - It defines when it should be created or removed

State inconsistency occurs because:

- The container may already be removed by systemd
- Or systemd may attempt to recreate it
- Or podman start may fail because the container no longer exists

This calls controllers ownership conflict

- The correct operational rule

```
systemctl start container.service
systemctl stop container.service
systemctl restart container.service
```

- With these command fix the ownership conflict.

Week 2/3 DB & HA cluster/ Monitoring Layer

Project Progress and Milestones

- **Zabbix Server**
- **PostgreSQL Primary and Replica**
- **VIP (Virtual IP)**
- **Grafana Dashboard**
- **Graylog**

Accomplishment

During this week, I created the fundamentals of the monitoring layer and the DB\Ha cluster and did tasks and here is the list of tasks:

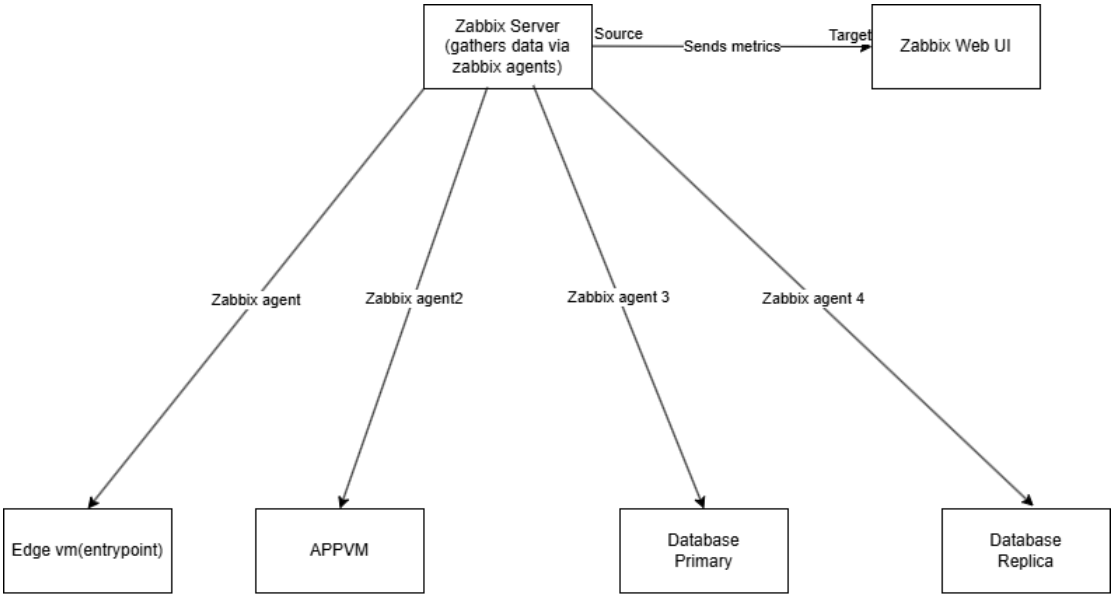
1)Building and deploying Zabbix server, Zabbix agents for each vm, added an item and a trigger for checking the Edge VM(entrypoint), then failover test

2)Database Primary and Replica can speak to each other and do their jobs, which I specified (Completed)

3)Virtual IP that creates a master and a backup/slave, and when those terms will change

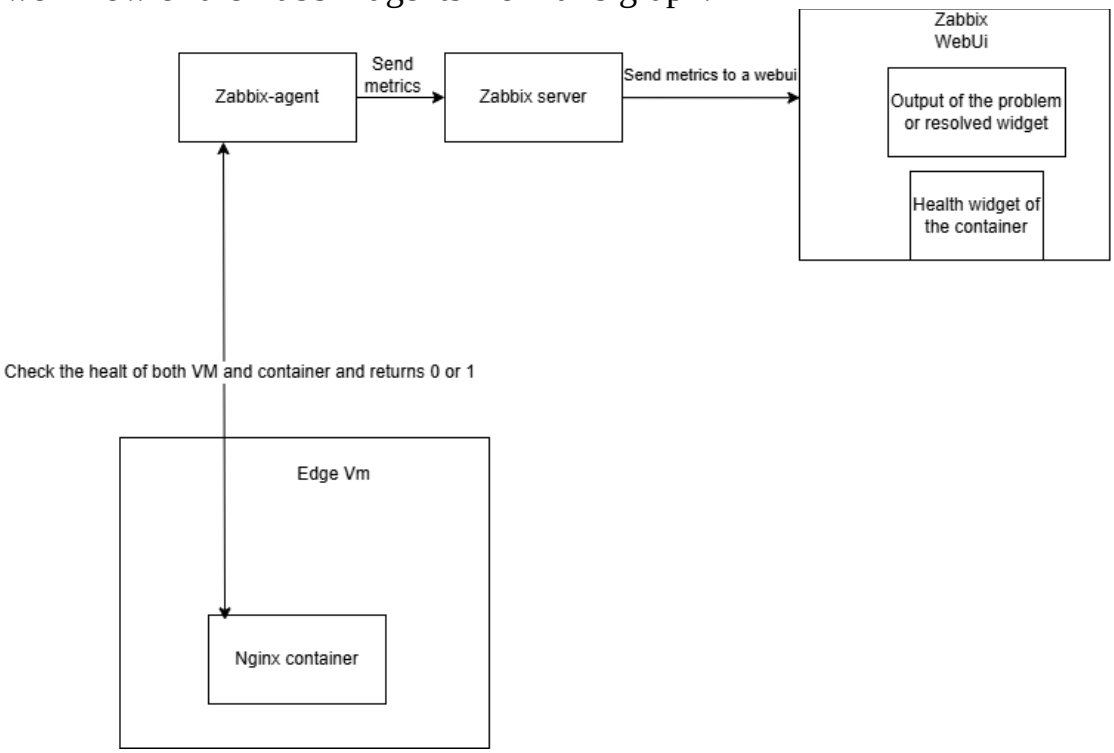
Now, for a better understanding, let's see the workflow of these.

Workflow of the Zabbix server



Here is the Zabbix server workflow. The server sends metrics and gathers info to see what happens in the VMs, what the hardware health is, CPU utilisation, and container health.

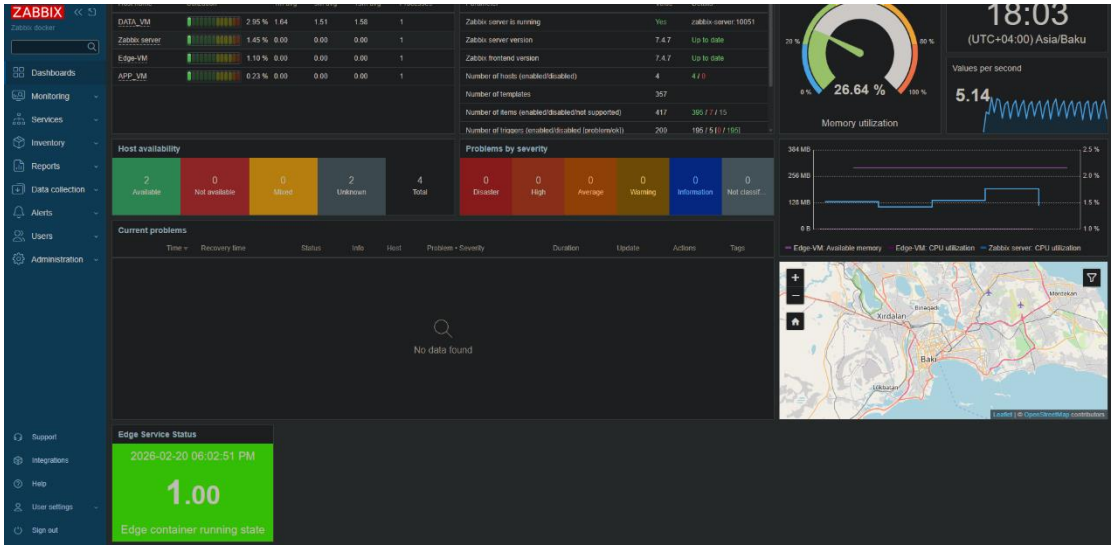
For more information about the Zabbix server, need to understand the workflow of the Zabbix agents from this graph.



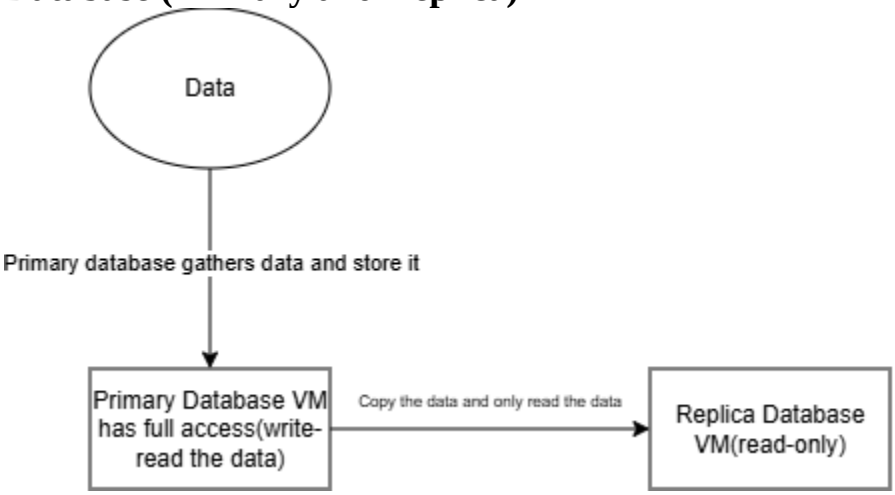
As we see from this picture, the Zabbix agent works like a “clerk” and do health check of 2 things:

1. Health of the VM

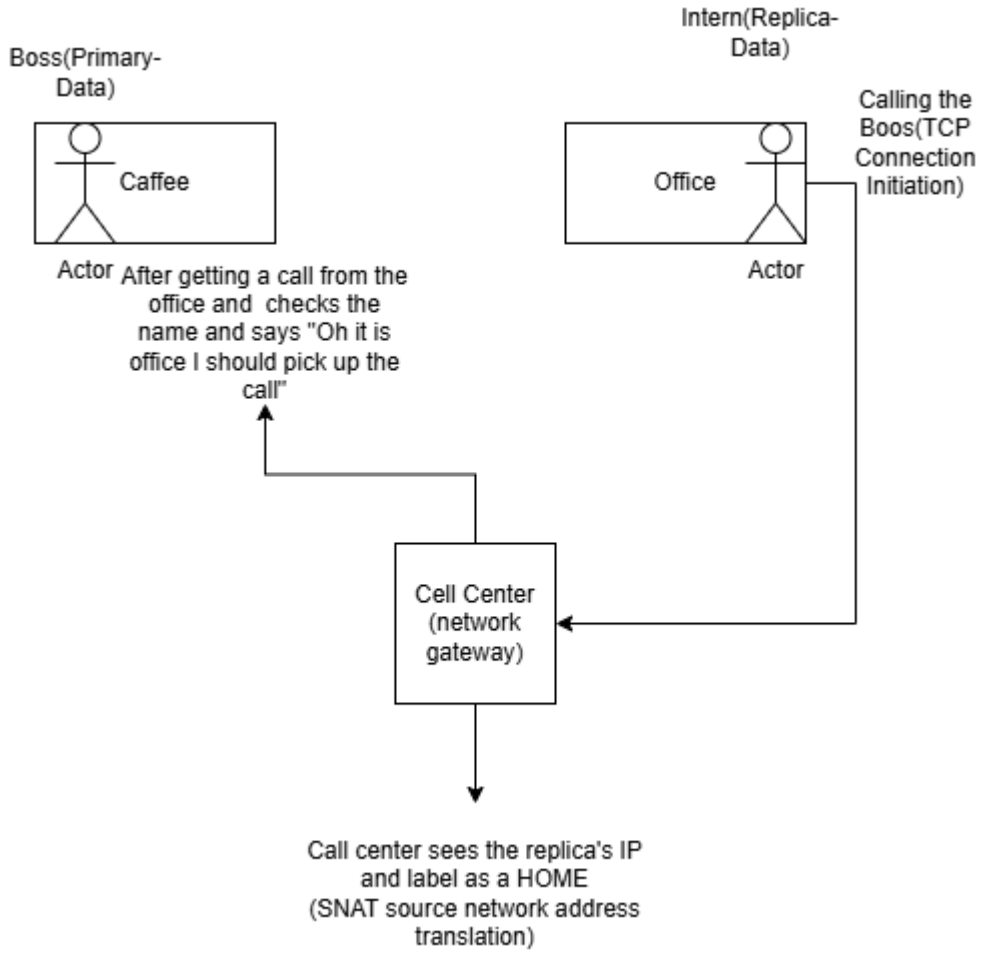
2. Health of the container/service



Database (Primary and Replica)



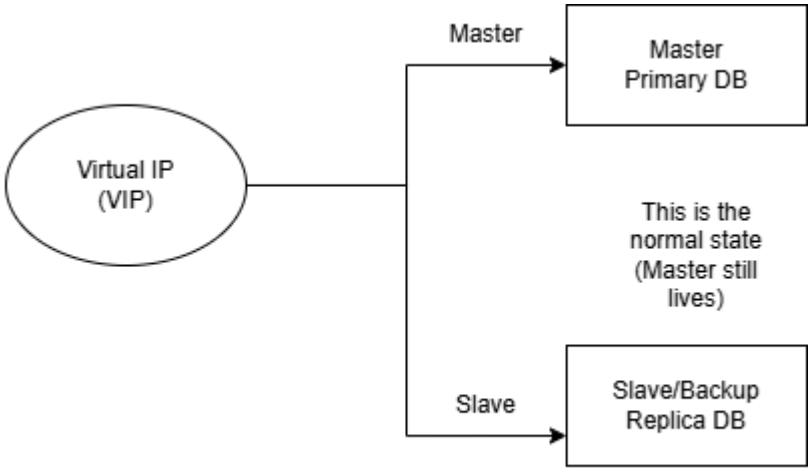
This picture shows the basic workflow of data writing/reading and sending the data to the backup database. If we look deeper, we can see the communication between the Primary and Replica databases.

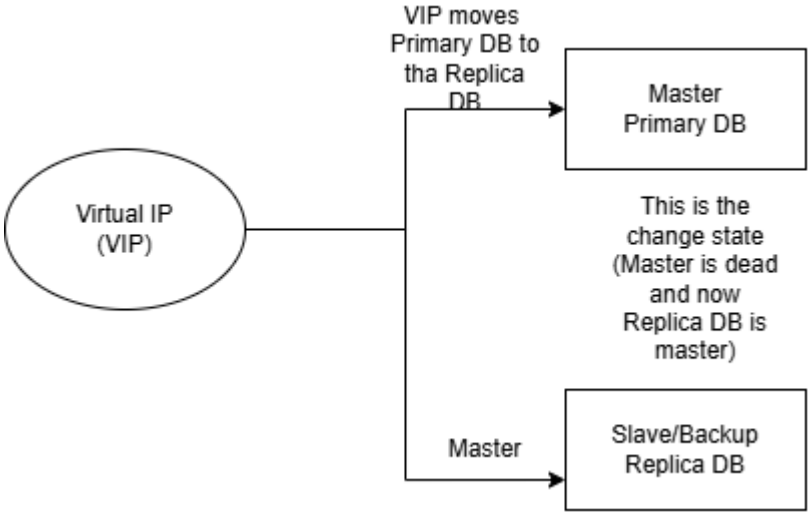


With the help of this image, now we can easily understand that when in Primary, we insert into the table some new values, let’s say "Hello", the replica instantly receives this text. It works like this: "BOSS-INTERN" analogy. The intern always "calls" the boss to ask for any new updates, and each update that happens boss instantly sends this "update" to the intern.

VIP (Virtual IP)

For creating a VIP, I used the keepalived service, and this is the workflow of a VIP.





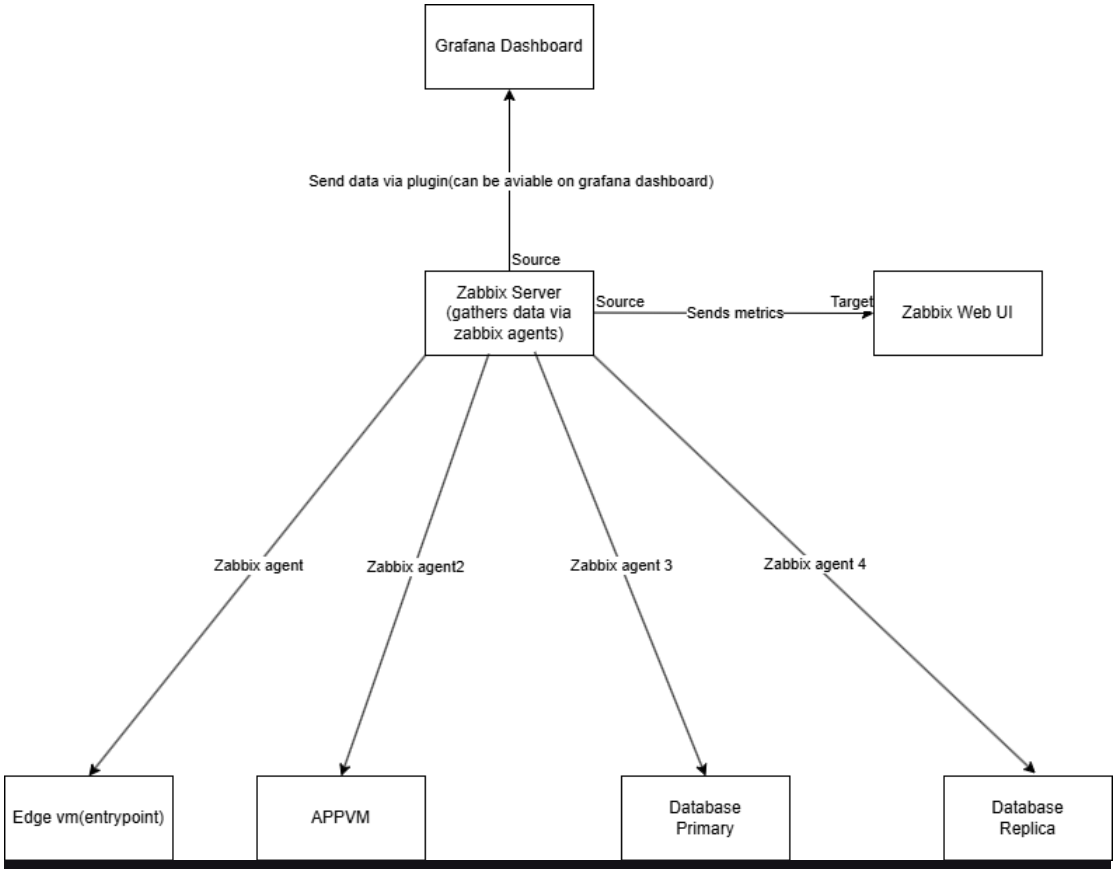
What happens if we don't use VIP/keepalived service:

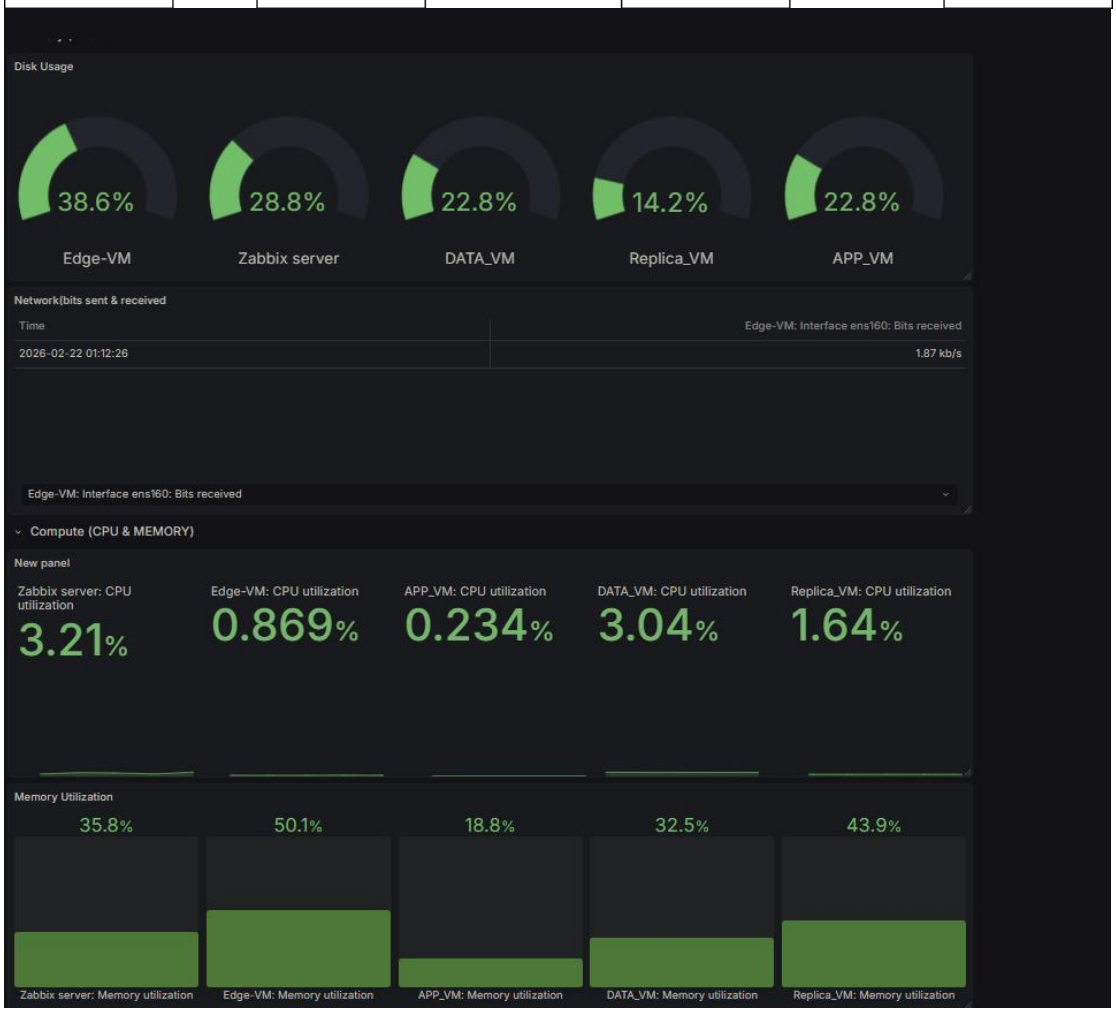
1. Downtime: When Primary is dead, it needs to promote the replica manually, and that causes downtime. In other words, this problem is called SPOF(single point of failure)
2. Split Brain Risk: Even if we promote the replica manually, after starting the primary, it causes "Split Brain". Think like this, in one cafe there are 2 bosses and gives different orders, and these cause conflicts.

Keepalived/VIP solves these problems, stabilizing MASTER/BACKUP state. When the primary dies, VIP automatically moves and gives the 'MASTER' state to a Replica.

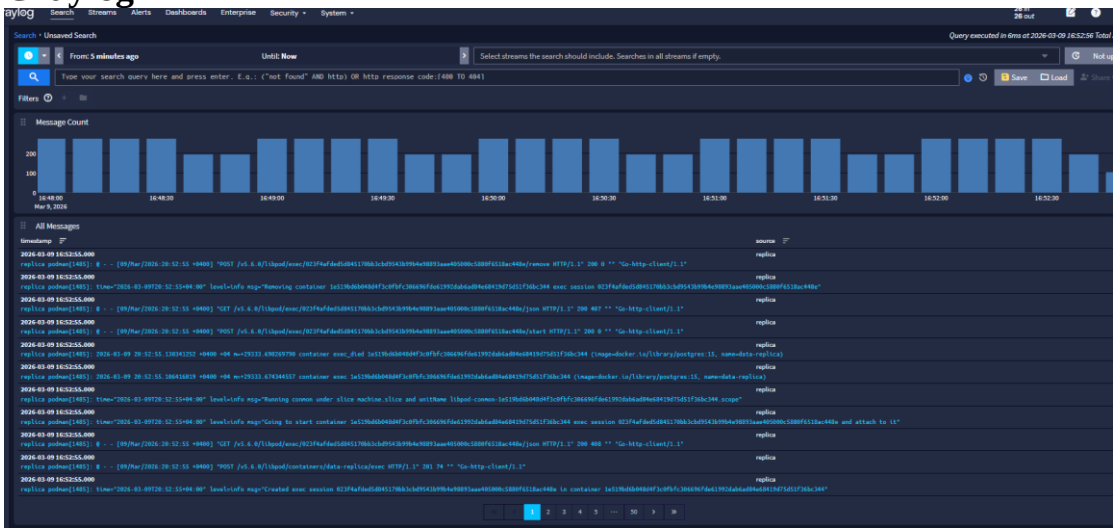
Grafana Dashboard

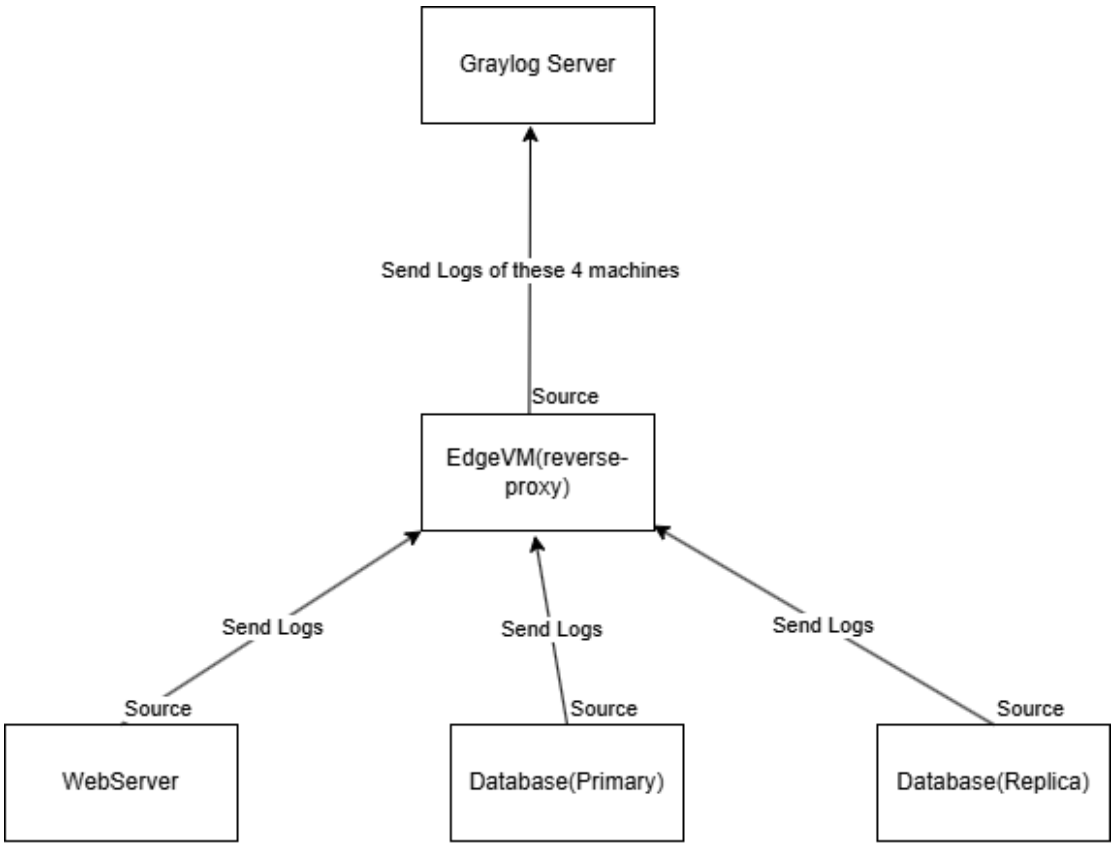
This service helps to monitor all systems, seeing the network throughput, visualising log flow, etc. It helps to centralise and control all management services on one service.


Grafana Dashboard
Send data via plugin(can be aviable on grafana dashboard)
Source
Zabbix Server (gathers data via zabbix agents)
Source
Sends metrics
Target
Zabbix Web UI
Zabbix agent
Zabbix agent2
Zabbix agent 3
Zabbix agent 4
Edge vm(entrypoint)
APPVM
Database Primary
Database Replica


Disk Usage
38.6%
Edge-VM
28.8%
Zabbix server
22.8%
DATA_VM
14.2%
Replica_VM
22.8%
APP_VM
Network(bits sent & received
Time
Edge-VM: Interface ens160: Bits received
2026-02-22 01:12:26
1.87 kb/s
Edge-VM: Interface ens160: Bits received
Compute (CPU & MEMORY)
New panel
Zabbix server: CPU utilization
3.21%
Edge-VM: CPU utilization
0.869%
APP_VM: CPU utilization
0.234%
DATA_VM: CPU utilization
3.04%
Replica_VM: CPU utilization
1.64%
Memory Utilization
35.8%
50.1%
18.8%
32.5%
43.9%
Zabbix server: Memory utilization
Edge-VM: Memory utilization
APP_VM: Memory utilization
DATA_VM: Memory utilization
Replica_VM: Memory utilization

Graylog





So this is how Graylog works. As we know, these 3 servers has internal network that means they can't send logs directly to the Graylog Server. But I have EdgeVM that has 2 legs (.50 subnet for connecting the internal network, .11 subnet for reaching the network the NAT). In the end, these 3 VMs that have an internal network can send logs to the Graylog through the EdgeVM.

Challenges and Issues

- **Invert data about the Zabbix-agent of the Edge VM**

podman run -d --name zabbix-agent-edge --restart=always --network host --privileged -e TZ="Asia/Baku" -v /:/host:ro -v /etc/zabbix:/etc/zabbix:ro -v /run/podman/podman.sock:/var/run/docker.sock -v /var/run/dbus/system_bus_socket:/var/run/dbus/system_bus_socket docker.io/zabbix/zabbix-agent2:latest -c /etc/zabbix/zabbix_edge.conf

Here is the meaning of all the flags and the workflow of this command:

-d: detached container will work in the background

--name: name of the container

--restart=always: even after a reboot, the container still lives

--network host: the container will connect to the host's interface

--privileged: grants full access

-e TZ=" Asia/Baku": adding an environment timezone of Baku

-v /:/host:ro: mount the root to a container's host and give read-only permission

-v /etc/zabbix:/etc/zabbix:ro: mount the local Zabbix conf directory into the container

-v /run/podman/podman.sock:/var/run/docker.sock: with this mount, Zabbix agent can speak in podman(because normally Zabbix agent has a built-in plugin for docker so it can't "natively speak Podman" a.k.a socket spoof

-v /var/run/dbus/system_bus_socker:/var/run/dbus/system_bus_socket : with that mount I built a nervous system. With that line now, the agent can look only at a specific systemd service(shortly connects the container to the host's inter D-Bus system. Also give 644 perm to /etc/Zabbix/zabbix_Data.conf(the rulebook to read the conf file and

666 perm to /run/podman/podman.sock to read write perm(with that it secure "walkie-talkie")

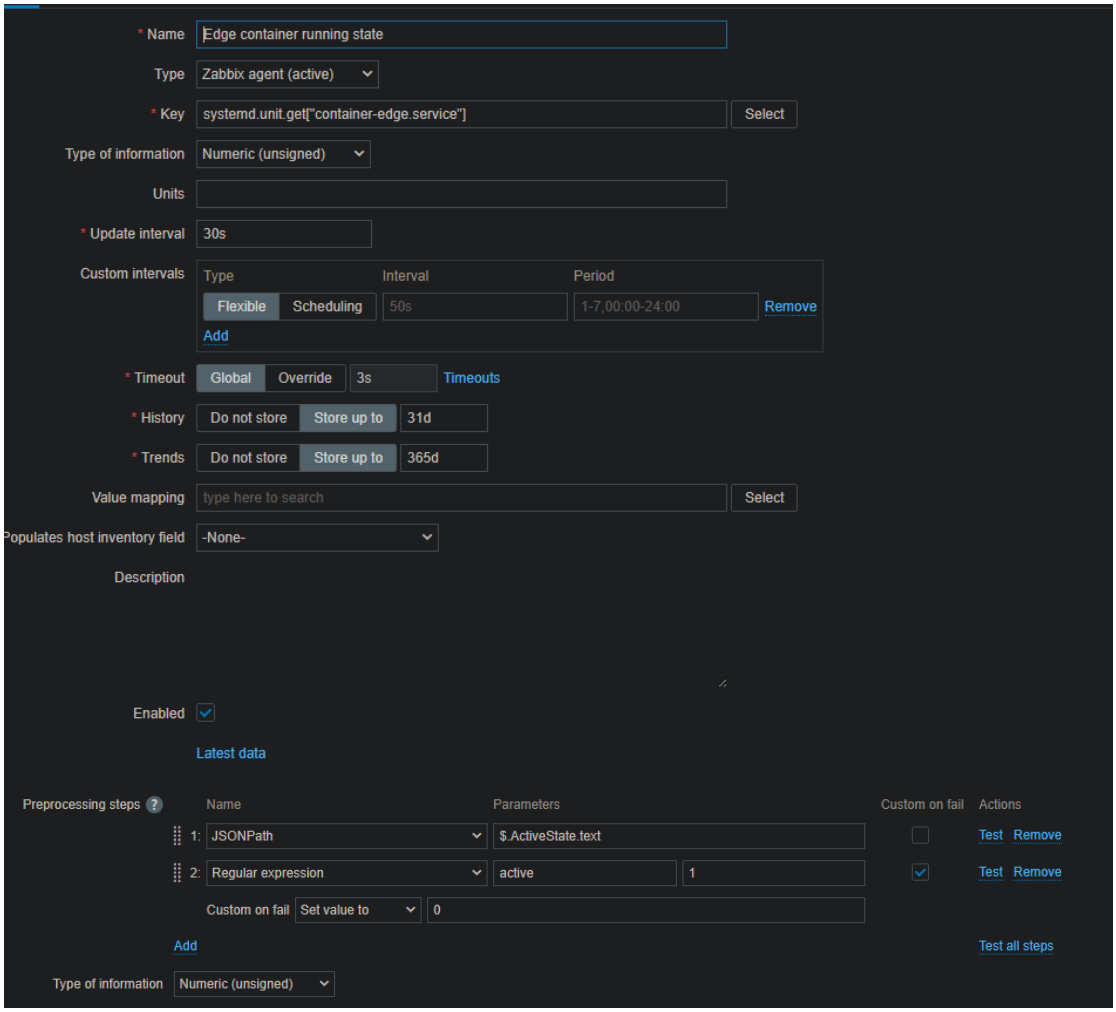
docker.io/Zabbix/zabbix-agent2:latest: this is the path from where it should be pulled

-c /etc/Zabbix/zabbix_edge.conf: with that command, Zabbix reads this conf file and obeys the rules or conf written in there, a.k.a it instructs the agent to load and apply

Workflow of the Zabbix agent and solution of the inverted output info (0,1):

From that command, I deployed a smart Zabbix agent that now checks from where, also in the Zabbix web ui I added a JSON path that pulls raw data, which I specifically defined the service and checks its health. As we know, the Zabbix agents love numbers instead of words, so after checking the health, and if the agent sees the "active"(it acts as a filter), it turns to a 1 or vice versa. Also, using the JSON path helps to check faster. Think of it like market aisles. In a supermarket, there are a lot of aisles, and let's suppose you want some chips, you need to check the "Snacks" aisle. After checking that aisle, you found the chips, and you're happy and ready to go home. JSON path works just like this: it checks inside the specified systemd service that checks 500 lines or more text, but with "$.ActiveState.text", it checks the word active from this "aisle", and after seeing this word, it is happy and ready to give "1" output or vice versa.

```
Hostname=Edge-VM
ServerActive=192.168.11.146
# Security Settings
TLSConnect=psk
TLSAccept=psk
TLSPSKIdentity=PSK_EDGE
TLSPSKFile=/etc/zabbix/zabbix_agent2.psk
# Targeted Security: Only allow your specific container check
AllowKey=system.run[*]
# Plugin Settings (Point to the socket)
Plugins.Docker.Endpoint=unix:///var/run/docker.sock
# Performance: Check for new items every 30s
RefreshActiveChecks=30
# Logging
LogType=console
```



- VIP

1) Code exited with 127 error
2) SELinux exec_died error
3) VIP cannot move to the replica
4)Reboot resistible and more solid

Solutions:
1) So I edited my whole script

```
#!/bin/bash
if /usr/bin/podman --url unix:///run/podman/podman.sock exec data-primary pg_isready -U postgres > /dev/null 2>\&1; then
   exit 0
else
   exit 1
fi
```

edited version

```
#!/bin/bash
if podman --url unix:///run/podman/podman.sock exec data-primary
pg_isready -U postgres > /dev/null 2>\&1; then
   exit 0
else
   exit 1
fi
```

Workflow of this script:

if

- **The Logic:** This tells Bash to evaluate the exit code of the command that follows it. If the command succeeds (returns a 0), it will execute the then block. If it fails (returns a 1 or anything else), it skips to the else block.

podman

- **The Logic:** You are calling the container engine program to do the work.

--url unix:///run/podman/podman.sock

- **The Logic:** This is the communication wire. Because Keepalived is running this script in the background as a system service, it doesn't have your normal user environment. This explicitly tells Podman, "Do not guess where the engine is; talk directly to this exact Unix socket file to send your commands." (This is why your SELinux and permissions fixes were so critical).

exec

- **The Logic:** The Podman subcommand that means "execute." It tells Podman to pierce through the isolation of the container and run a command *inside* of it, rather than doing something to the outside of the container (like stopping or starting it).

data-primary

- **The Logic:** The exact name of the container you want to reach into.

pg_isready

- **The Logic:** This is the actual payload. It is a built-in PostgreSQL utility. Instead of running a complex SQL query, this tool simply knocks on the database port and asks, "Are you alive and accepting connections?"

-U postgres

- **The Logic:** An argument for pg_isready. It specifies that the knock on the door should be done as the database user named Postgres.

> /dev/null

- **The Logic:** The silencer. Normally, **pg_isready** prints text to the screen (like "192.168.50.100:5432 - accepting connections"). Keepalived doesn't care about reading text; it only cares about the math (the exit code). This takes standard output (**stdout**) and throws it into /dev/null, which is the Linux trash can.

2>&1

- **The Logic:** The error silencer. In Linux, standard errors (**stderr**, designated by the number **2**) are printed separately from standard output (**stdout**, designated by **1**). This command says, "Take standard error **(2)** and redirect it **(>)** to the same place standard output is going **(&1**)." So, if the database connection fails and screams "Connection refused!", that error text also goes straight into the trash can.

; then

- **The Logic:** The closer. The semicolon marks the end of the command you are testing. The **"then"** tells Bash, "If everything before this semicolon succeeded quietly and returned a 0, proceed to the next line."

This script translates to:
Bash, strictly use this socket to tell Podman to reach inside data-primary and ask the database if it is awake. Throw away any text it replies with. If the database is awake, proceed.

After changing my container into a system service i need to edit the absolute path of podman
After this edit, the 127 error is gone and worked properly.

2) One of the "headache" problems was SELinux, so after researching the soluion was so easy
After editing and applying podman socket into my script(which checks the health of my primary-data service/container), I wrote these commands on both VMs

1) Enabled the podman socket and gave 660 perm

2) Then restorecon -v /usr/local/bin/check_pg.sh. With these 2 steps, the exec_died error is solved

3) The command that helps to solve and works properly is mcast_src_ip
So it literally works when the primary is dead, it forces the replica to communicate with the master and tell it to "change your state as a master, my VIP is coming through, take it"

```
    vrrp_script chk_pg {
        script "/usr/local/bin/check_pg.sh
        interval 1
        fall 3
        weight -20
        user root

    }

vrrp_instance VI_1 {
    state MASTER
    interface ens160
    virtual_router_id 51
    mcast_src_ip 192.168.50.2
    priority 100
    advert_int 1
    authentication {
        auth_type PASS
        auth_pass 2222
    }
    virtual_ipaddress {
        192.168.50.100
    }
    track_script{
        chk_pg
    }
}
```

4) The solution is easy: turn to the systemd service

1)It solve auto-start problem

2)Cluster brain connection: It is much harder to maintain a predictable state when a container is not managed by a service. Now, Keepalived is checking a formal system service instead of a random, isolated process.

Think of it like a freelance chef vs. a salaried restaurant chef. Both might arrive and leave at the same time, but the freelancer won't come back the next day unless you manually call them. Raw Podman containers are like the freelancer—because Podman is daemonless, the containers have no persistent manager in the background to restart them if they fail or if the server reboots. Systemd acts as the restaurant manager, ensuring the chef is always at their station so Keepalived gets an accurate health check.

3)Clean kill and easier to trigger a failover.

With "sytsemctl stop container-data-primary.service", you pointed out which container should "die" now.

- **Resolving Containerised Log Stream Failures**

Problem Statement

The EdgeVM (Source) can successfully ping or reach the Monitoring VM (Host), but logs fail to reach the Graylog container. The connection is explicitly refused despite the service appearing to be "up."

After deploying Graylog and adding this text into a config file, I get an error that EdgeVM can communicate with Graylog VM, but can't send logs to this container.

Solving this problem

1) Checking firewall and SELinux, after adding a specific port that I launch the input with, the port requires adding firewall for both tcp and udp protocol of this port

2) after that, testing and seeing the logs via tcpdump and testing via netcat and ss -tulpn | grep command

3) After getting "bad news" from these command I recognised something. When I deploy the graylog for the first time, I bind ports only to UDP, but I launch the input with tcp and that's why it gets a conflict and connection refused.

4) To solve that problem, it needs to redeploy Graylog with proper ports and protocol:

First, delete the container, then its volume(Delete the container, but keep the volume if you want to retain your history. Only delete the volume for a 'factory reset'.)

Second, deploy again but add -p 1514:1514/tcp -p 1514:1514/udp -p 1515:1515/tcp, because the first port is for the general log setting and the second port is for the communication, which shortly prevents "Protocol Locked."

And with that, our VM can automatically stream data to the graylog.

Week 3/4 DNS/Ansible/Full Automation

During this fortnight, first I built the DNS and wrote its configuration. After configuration, I had a question: “What if I have another DNS server or 2-3 more DNS servers? Do I need to create a config for each server?” While I was asking myself these questions, I found Ansible, the Automation tool. In these weeks, I built the automation architecture for my system, wrote playbooks and troubleshooted the problems.
First, I needed to build architecture:

```
.
├── ansible.cfg
├── host_vars
│   └── web
│       └── main.yml
├── inventory
│   ├── group_vars
│   └── production
├── playbooks
│   ├── deploy_containers.yml
│   └── patch.yml
├── roles
│   ├── common
│   │   └── tasks
│   │       └── main.yml
│   ├── db
│   ├── dns-server
│   │   ├── handlers
│   │   │   └── main.yml
│   │   ├── tasks
│   │   │   └── main.yml
│   │   └── templates
│   │       ├── monitoring.local.forward
│   │       ├── monitoring.local.reverse.11
│   │       ├── monitoring.local.reverse.50
│   │       └── named.conf.j2
│   ├── firewall
│   │   └── tasks
│   │       └── main.yml
│   ├── graylog_forward
│   │   ├── handlers
│   │   │   └── main.yml
│   │   ├── tasks
│   │   │   └── main.yml
│   │   └── templates
│   │       └── graylog.conf.j2
│   ├── podman_stack
│   ├── web
│   │   ├── files
│   │   │   └── template.zip
│   │   ├── handlers
│   │   │   └── main.yml
│   │   └── tasks
│   │       └── main.yml
│   └── zabbix_agent
└── site.yml
```

With that tree, I can automate DNS master/slave configuration, web automation, private mirror setup/local repo automation.

For DNS automation, I wrote this playbook(it's a list of tasks):

```yaml
---
    - name: Ensure BIND is installed
      package:
        name: bind
        state: latest

    - name: Configure named.conf from template
      template:
        src: named.conf.j2
        dest: /etc/named.conf
        owner: root
        group: named
        mode: '0640'
        # This prevents the playbook from breaking DNS if you missed a semicolon
        validate: 'named-checkconf %s'
      notify: restart named

    - name: Find existing slave zone files
      find:
        paths: /var/named/slaves
        patterns: "*"
      register: found_zones
      when: "'dns_slave' in group_names"

    - name: Delete the old zones (Ensures fresh sync from Master)
      file:
        path: "{{ item.path }}"
        state: absent
      loop: "{{ found_zones.files }}"
      when:
        - "'dns_slave' in group_names"
        - found_zones.matched > 0

    - name: Ensure Slave Directory has correct permissions
      file:
        path: /var/named/slaves
        state: directory
        owner: named
        group: named
        mode: '0770'
        setype: named_cache_t

    - name: Push All Zone Files to Master
      template:
        src: "{{ item }}"
        dest: "/var/named/{{ item }}"
        owner: root
        group: named
        mode: '0640'
        setype: named_zone_t

      loop:
        - monitoring.local.forward     # Forward zone
        - monitoring.local.reverse.11 # 1st reverse zone
        - monitoring.local.reverse.50 # 2nd reverse zone
      when: "'dns_master' in group_names"
      notify: restart named

    - name: Restore SELinux context
      command: restorecon -Rv /var/named
      changed_when: false

    - name: Ensure named service is started and enabled
      service:
        name: named
        state: started
        enabled: yes
```

For ensure the workflow, I also need templates (config files):

```
$TTL 86400
@       IN SOA  ns1.monitoring.local. admin.monitoring.local. (
                {{ ansible_date_time.epoch }}; Dynamic Serial (UNIX Timestamp)
                28800      ; Refresh (each 8h)
                7200       ; Retry (each 2h)
                604800     ; Expire(1 week)
                86400      ; Minimum TTL (1 day)
)
@        IN NS   ns1.monitoring.local.
@        IN NS   ns2.monitoring.local.

; --- Infrastructure Hosts (.11 Subnet) ---
ns1             IN A   192.168.11.146    ; Monitoring-server VM
ns2             IN A   192.168.11.150    ; Backup-DNS VM
edge            IN A   192.168.11.100    ; Edge VM
; --- Application & DB Hosts (.50 Subnet) ---
app             IN A   192.168.50.3      ; App VM
dbvm            IN A   192.168.50.2      ; DB VM
data-replica    IN A   192.168.50.6      ; Data-replica VM
```

```
$TTL 86400
@       IN SOA  ns1.monitoring.local. admin.monitoring.local. (
                {{ ansible_date_time.epoch }}; Dynamic Serial (UNIX Timestamp)
                28800      ; Refresh(each 8h)
                7200       ; Retry (each 2h)
                604800     ; Expire (1 week)
                86400      ; Minimum TTL (1 day)
)
; Name Servers
@       IN NS      ns1.monitoring.local.
@       IN NS      ns2.monitoring.local.

; PTR Records (IP to Name)
146     IN  PTR     ns1.monitoring.local.
150     IN  PTR     ns2.monitoring.local.
100     IN  PTR     edge.monitoring.local.
```

```
$TTL 86400
@       IN SOA  ns1.monitoring.local. admin.monitoring.local. (
                {{ ansible_date_time.epoch }}; Dynamic Serial (UNIX Timestamp)
                28800      ; Refresh
                7200       ; Retry
                604800     ; Expire
                86400 )    ; Minimum TTL

@       IN NS      ns1.monitoring.local.
@       IN NS      ns2.monitoring.local.

; PTR Records for 192.168.50.x
3       IN PTR     app.monitoring.local.
2       IN PTR     dbvm.monitoring.local.
6       IN PTR     data-replica.monitoring.local.
```

```
// ##################################################################
// # ANSIBLE MANAGED FILE - DO NOT EDIT MANUALLY ON THE SERVER      #
// ##################################################################

options {
    listen-on port 53 { 127.0.0.1; 192.168.11.146; 192.168.11.150; };
    directory       "/var/named";
    dump-file       "/var/named/data/cache_dump.db";
    statistics-file "/var/named/data/named_stats.txt";
    memstatistics-file "/var/named/data/named_mem_stats.txt";
    // Allow both subnets to query
    allow-query     { localhost; 192.168.11.0/24; 192.168.50.0/24; };
    allow-update    { none; };
    recursion yes;
    dnssec-validation yes;
    include "/etc/crypto-policies/back-ends/bind.config";
    managed-keys-directory "/var/named/dynamic";
    pid-file "/run/named/named.pid";
    session-keyfile "/run/named/session.key";
    version "not disclosed";
    hostname none;
};

logging {
    channel default_debug {
        file "data/named.run";
        severity dynamic;
    };
};

zone "." IN {
    type hint;
    file "named.ca";
};

// ================= FORWARD ZONE =================
zone "monitoring.local" IN {
    {% if 'dns_master' in group_names %}
    type master;
    file "monitoring.local.forward";
    # Allow transfer to the slave IP
    allow-transfer { 192.168.11.150; };
    also-notify { 192.168.11.150; };
    {% else %}
    type slave;
    file "slaves/monitoring.local.forward";
    # Pull from the master IP
    masters { 192.168.11.146; };
    {% endif %}
```

```
};

// ================= REVERSE ZONES =================
{% for subnet in ['11', '50'] %}
zone "{{ subnet }}.168.192.in-addr.arpa" IN {
    {% if 'dns_master' in group_names %}
    type master;
    file "monitoring.local.reverse.{{ subnet }}";
    allow-transfer { 192.168.11.150; };
    {% else %}
    type slave;
    file "slaves/monitoring.local.reverse.{{ subnet }}";
    masters { 192.168.11.146; };
    {% endif %}
};
{% endfor %}
```

In short, with the help of these 3 configurations, the play master and slave DNS can configure automatically and ensure working on all subnets (that is why, for the internal network, I wrote a reverse zone for the .50 subnet, for the other VMs that can reach the network wrote a zone for the .11 subnet). Also, for the slave DNS server, it automatically creates a slave directory and pushes all master zones to this directory. Also, with that configuration, we earned these types of securities:

Layer	Component	How it stops a Zero Day
ACL (Shield)	`allow-query`	Even if there is a Zero Day in the query-processing code, an attacker from the public internet **cannot send the packet** to trigger it because they are blocked by your IP list.
OS Firewall	`firewalld / iptables`	Blocks the traffic before it even reaches the BIND software.
SELinux	`named_zone_t`	**Crucial!** If a Zero Day allows a hacker to "break out" of the BIND process, SELinux will block them from touching your root files or password database.
Ansible	`package: state: latest`	Allows you to patch every server in your network in seconds the moment a fix is released.

The second automation

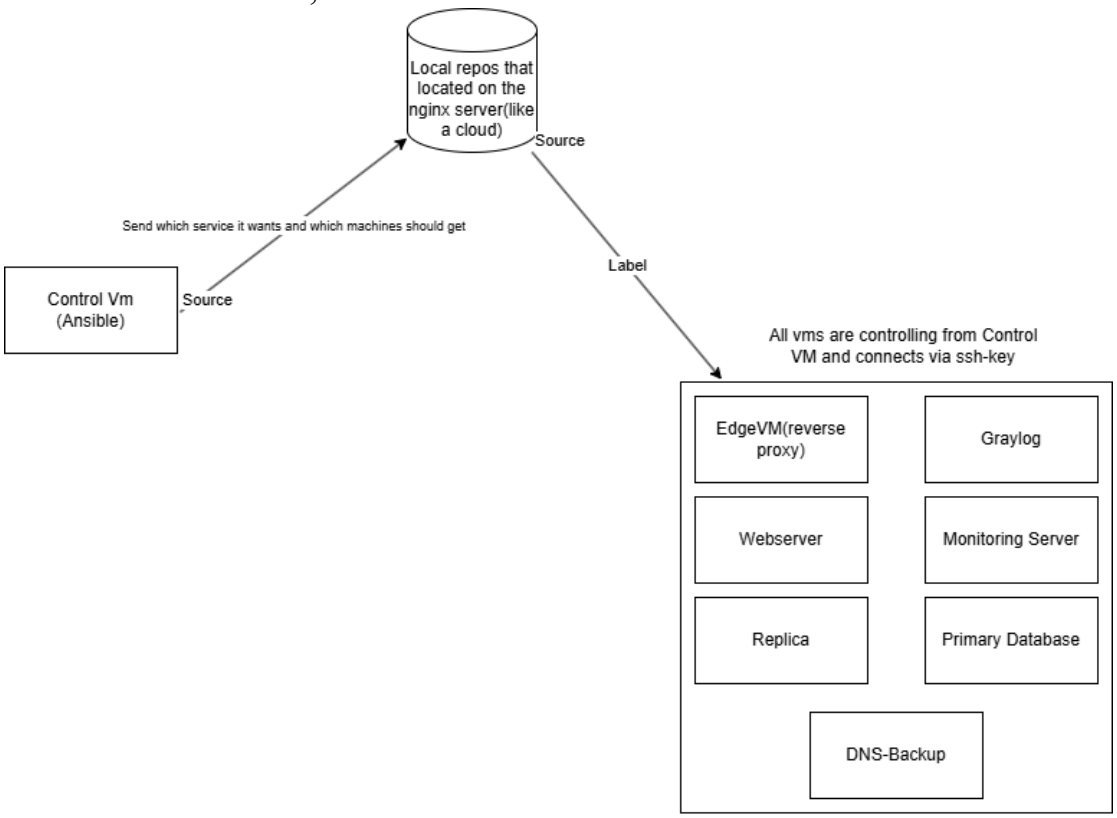
```yaml
# SECTION 1: EDGE VM ROUTING & MASQUERADE
- name: Configure Edge VM as NAT Gateway
  when: inventory_hostname in groups['edge']
  block:
    - name: Enable IP Forwarding in Kernel
      ansible.posix.sysctl:
        name: net.ipv4.ip_forward
        value: '1'
        sysctl_set: yes
        state: present

    - name: Enable Firewalld Masquerade
      ansible.posix.firewalld:
        masquerade: yes
        state: enabled
        permanent: yes
        immediate: yes

# SECTION 2: NETWORK CONFIG (Corrected for all VMs)
- name: Configure NetworkManager DNS Settings
  community.general.nmcli:
    # This automatically finds the right interface (ens160, eth0, etc.)
    conn_name: "{{ ansible_default_ipv4.interface }}"
    type: ethernet
    dns4:
      - 192.168.11.146
      - 192.168.11.150
    dns4_search:
      - monitoring.local
    # This is the "Ignore" checkbox from nmtui
    dns4_ignore_auto: yes
    # Gateways for the differents subnets
    gw4: >-
      {% if ansible_default_ipv4.address.startswith('192.168.50.') %}
      192.168.50.1
      {% elif ansible_default_ipv4.address.startswith('192.168.11.') %}
      192.168.11.2
      {% else %}
      omit
      {% endif %}
    state: present
  register: nm_change

- name: Force rewrite of resolv.conf
  ansible.builtin.shell: "nmcli connection up {{ ansible_default_ipv4.interface }}"
  when: nm_change.changed
```

```yaml
# SECTION 3: REPO SERVER BUILD (Only for the 'control' VM)
- name: Build Repository Server
  when: inventory_hostname == 'control'
  block:
    - name: Install Nginx
      dnf:
        name: nginx
        state: present

    - name: Configure Nginx for Directory Listing
      copy:
        dest: /etc/nginx/conf.d/repo.conf
        content: |
          server {
              listen 80;
              server_name 192.168.11.151;
              root /var/www/html;

              location /repo {
                  autoindex on;
                  autoindex_exact_size off;
                  autoindex_localtime on;
              }
          }

    - name: Remove default Nginx welcome configuration
      file:
        path: /etc/nginx/conf.d/welcome.conf
        state: absent

    - name: Create Repo Directory
      file:
        path: /var/www/html/repo
        state: directory
        mode: '0755'
        setype: httpd_sys_content_t
    - name: Mount ISO and add to /etc/fstab (Reboot-Persistent)
      mount:
        path: /var/www/html/repo
        src: /dev/sr0
        fstype: iso9660
        opts: 'ro,loop,context="system_u:object_r:httpd_sys_content_t:s0"'
        state: mounted

    - name: Configure SELinux and Firewall
      block:
        - name: Allow Nginx to read from mount
          seboolean:
            name: httpd_read_user_content
                                                                    91,0-1
```

```yaml
            name: httpd_read_user_content
            state: yes
            persistent: yes

        - name: Open HTTP Firewall Port
          firewalld:
            service: http
            permanent: yes
            state: enabled
            immediate: yes

    - name: Restart and Enable Nginx Service
      service:
        name: nginx
        state: restarted
        enabled: yes

    - name: Verify AppStream is Live
      uri:
        url: http://192.168.11.151/repo/AppStream/
        status_code: 200
      register: appstream_check
      until: appstream_check.status == 200
      retries: 5
      delay: 5
      when: not ansible_check_mode

    - name: Verify BaseOS is Live
      uri:
        url: http://192.168.11.151/repo/BaseOS/
        status_code: 200
      register: baseos_check
      until: baseos_check.status == 200
      retries: 5
      delay: 5
      when: not ansible_check_mode

# SECTION 4: TIMEZONE (Runs on EVERY VM at the end)
- name: Set system timezone
  community.general.timezone:
    name: Asia/Baku


# SECTION 5: MINI-CLOUD CONNECTOR (Runs on EVERY VM except control)
- name: Point all VMs to Mini-Cloud
  when: inventory_hostname not in groups['manager']
  block:
    - name: Disable all internet-facing repositories
      ansible.builtin.shell: "dnf config-manager --set-disabled '*'"
      failed_when: false
      changed_when: false

    - name: Add Mini-Cloud Repositories
      ansible.builtin.yum_repository:
        name: "{{ item.id }}"
        description: "{{ item.name }}"
        baseurl: "http://192.168.11.151/repo/{{ item.path }}"
        enabled: yes
        gpgcheck: no
      loop:
        - { id: 'local-baseos', name: 'Mini-Cloud BaseOS', path: 'BaseOS' }
        - { id: 'local-appstream', name: 'Mini-Cloud AppStream', path: 'AppStream' }

    - name: Refresh DNF Cache
      ansible.builtin.command: dnf clean all
      changed_when: false

    - name: Generate DNF Cache
      ansible.builtin.command: dnf makecache
      changed_when: false
```

1) So, after creating DNS servers, I need to send to all my machines' /etc/resolv.conf
2) Also, it automatically configures the network on all machines (even changes gateways for specific subnets). For the .50 subnet, the gateway is .50.1; for the .11 subnet, the gateway is .11.2.
3) On Control VM (here ansible service is working, and I'm writing playbooks on this VM), it creates local repositories, but mounted on the nginx service (private mirror, but I called it mini cloud)
4) At the end of the playbook, after creating the local repo, it also unregisters all VMs and "connects" to "mini cloud". So, I can download any packages or services from this VM for all VMs or whatever I want. It also helps to bypass the "register"

This is how it works, the "mini cloud"



The third automation that I've created is Graylog Forwarding:

```yaml
---
# Graylog Automation: Ensures logs flow from VM to Edge to Graylog
- name: Allow Rsyslog to connect to custom port 1515 (Permanent)
  community.general.seport:
    ports: 1515
    proto: tcp
    setype: syslogd_port_t
    state: present

- name: Enable NIS boolean for broader network compatibility
  ansible.posix.seboolean:
    name: nis_enabled
    state: yes
    persistent: yes

- name: Ensure rsyslog spool directory exists
  ansible.builtin.file:
    path: /var/lib/rsyslog
    state: directory
    owner: root
    group: root
    mode: '0700'

- name: Configure Rsyslog to forward to Graylog
  ansible.builtin.template:
    src: graylog.conf.j2
    dest: /etc/rsyslog.d/99-graylog.conf
    owner: root
    group: root
    mode: '0644'
  notify: reload rsyslog

- name: Ensure ports are enabled on firewalld
  ansible.posix.firewalld:
    port: "{{ item }}"
    state: enabled
    permanent: yes
    immediate: yes
  loop:
    - 1514/udp
    - 1514/tcp
    - 1515/tcp

- name: Ensure rsyslog service is started and enabled
  ansible.builtin.service:
    name: rsyslog
    state: started
    enabled: yes
```

As we talked about, the machines that have an internal network send logs to Graylog. With this playbook, it automatically does this configuration. For a better understanding of how this playbook works and how it knows the specific Graylog configurations, etc. The answer is simple. Templates. So, on Ansible, the templates are game changers. Because it really gives an “instruction” on where it should be changed or added.

```
# THE INPUT: This tells the Edge VM to listen on the ports you opened
{% if inventory_hostname in groups['edge'] %}
module(load="imtcp")
input(type="imtcp" port="1515")

module(load="imudp")
input(type="imudp" port="1514")
{% endif %}

# Centralized Logging Forwarder Configuration
# Forward all logs to Graylog over reliable TCP transport
*.* action(
    type="omfwd"
    {% if inventory_hostname in groups['edge'] %}
    target="192.168.11.148"
    {% else %}
    target="192.168.50.1"
    {% endif %}

    port="1515"
    protocol="tcp"

    action.resumeRetryCount="-1"
    action.reportSuspension="on"
    action.reportSuspensionContinuation="on"

    queue.type="linkedlist"
    queue.size="10000"
    queue.filename="graylog_queue"
    queue.spoolDirectory="/var/lib/rsyslog"
    queue.maxdiskspace="1g"
    queue.saveonshutdown="on"
)
```

The third automation is for web server configuration (adding the CSS template, then restarting it). It simply extracts the template and pushes it to the web server.

```yaml
---
- name: 1. Ensure unzip is installed
  ansible.builtin.package:
    name: unzip
    state: present
  become: true

- name: 2. Fix permissions on /root so Nginx can pass through
  ansible.builtin.file:
    path: "/root"
    mode: "0755"
  become: true

- name: 3. Create the parent directory
  ansible.builtin.file:
    path: "/root/nginx-html"
    state: directory
    mode: "0755"
  become: true

- name: 4. Extract the zip into the directory
  ansible.builtin.unarchive:
    src: "template.zip"
    dest: "/root/nginx-html"
    remote_src: no
  become: true

- name: 5. Move files out of the subfolder and into the root
  ansible.builtin.shell: |
    mv /root/nginx-html/templatemo_461_workforce/* /root/nginx-html/
    rm -rf /root/nginx-html/templatemo_461_workforce
  args:
    removes: "/root/nginx-html/templatemo_461_workforce"
  become: true

- name: 6. Set SELinux context and permissions recursively
  ansible.builtin.file:
    path: "/root/nginx-html"
    setype: "container_file_t"
    owner: "root"
    group: "root"
    mode: "0755"
    recurse: yes
  become: true
  notify: reload container app
```

During this automation learning session, I have learned that labelling the .yml files, building the correct hierarchy and architecture, is so important while working with Ansible.

Project Conclusion

This project successfully demonstrates the design and implementation of a small-scale automated infrastructure environment. Key components such as DNS services, installing services via mini cloud, monitoring and logging systems, centralised full-scale monitoring, and containerised services were deployed and integrated into a functional platform. Also, adding the keepalived service creates a DB & HA cluster.
The implementation of master -slave DNS architecture enabled reliable internal name resolution, while container-based monitoring and logging solutions provided improved visibility into system operations.

From my perspective, automation was central to the project. By using Ansible, system configuration and service deployment were automated across multiple virtual machines, improving consistency and reducing human interaction. Throughout the project, several challenges related to DNS configuration, container networking, container deployment, Zabbix wrong data showings, and an automation design were encountered and resolved.
In the end, the project provided me valuable hands-on experience with infrastructure automation, system administration, and network configuration, while demonstrating the practical benefits of automated infrastructure management.